Retrieval-Augmented Generation (RAG) 📚

Day – 15

Retrieval-Augmented Generation (RAG) is a technique that enhances the responses of Large Language Models (LLMs) by connecting them to an external knowledge base. It allows the model to "look up" relevant information before answering a question, making its responses more accurate, up-to-date, and trustworthy.

The Problem: LLMs Don't Know Everything

LLMs have two main limitations:

1. Knowledge Cutoff: Their knowledge is frozen at the time of their training. They don't know about events that happened after their training data was collected.
2. Hallucinations: They can sometimes invent facts or provide incorrect information when they don't know the answer.

RAG was designed to solve these problems by giving the model a way to access current and specific information.

Analogy: The Open-Book Exam 📖

- A traditional LLM is like a student taking a closed-book exam. It can only use the information it has memorized.
- A RAG-powered LLM is like a student taking an open-book exam. It can consult a textbook (the external knowledge base) before writing down its answer, leading to a much more accurate and detailed response.

How RAG Works: A Two-Step Process

RAG combines the power of information retrieval (like a search engine) with the text generation capabilities of an LLM.

Step 1: Retrieval (The "Look-up" Phase)

This step finds the relevant information.

1. **User Prompt:** The user asks a question (e.g., "What were our company's Q2 sales goals?").
2. **Search Query:** The system converts the user's prompt into a search query.
3. **Vector Database:** This query is used to search a specialized database called a vector database. This database contains your company's documents, reports, or any other private data, which has been pre-processed and stored as numerical representations called embeddings.

4. **Relevant Chunks:** The database finds and retrieves the pieces of text ("chunks") that are most semantically similar to the user's query. For our example, it would retrieve the section of a sales report detailing the Q2 goals.

Step 2: Augmentation and Generation (The "Answer" Phase)

This step uses the retrieved information to create a response.

1. **Augmented Prompt:** The system creates a new, more detailed prompt. It combines the original user question with the relevant text chunks retrieved from the database.
2. **LLM Response:** This augmented prompt is sent to the LLM. The model now has all the context it needs to answer accurately.
3. **Final Answer:** The LLM generates a final, human-readable answer based *only* on the provided context. For our example, it would respond: "According to the Q2 sales report, the sales goals were..."

Key Benefits of RAG

- **Reduces Hallucinations:** The model answers based on the provided documents, not just its internal memory, making it far less likely to invent information.
- **Access to Current Data:** You can constantly update your vector database with new information, allowing the LLM to provide up-to-the-minute answers without needing to be retrained.
- **Increased Trust and Transparency:** RAG systems can cite their sources, allowing users to verify where the information came from.
- **Cost-Effective:** It is much cheaper and faster to update a knowledge base than to fine-tune or retrain an entire LLM.

The Goal

Our goal is to create a script that can answer questions about a local code file named my_app.py.

- **User Question:** "How do I add two numbers using the app's functions?"
- **Knowledge Source:** A local file named my_app.py.
- **Desired Answer:** "You can use the add(a, b) function. It takes two numbers, a and b, as input and returns their sum."

```
import os

from langchain.llms import OpenAI
```

```
from langchain.document_loaders import TextLoader
from langchain.text_splitter import CharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA
# --- 1. SETUP: Load API Key ---
# Make sure your OpenAI API key is set as an environment variable
# os.environ["OPENAI_API_KEY"] = "sk-..."
# This is the local data our RAG system will use.
code_content = """
# my_app.py: A simple calculator application
def add(a, b):
    \"\"\"This function adds two numbers and returns the result.\"\"\"
    return a + b
def subtract(a, b):
    \"\"\"This function subtracts the second number from the first.\"\"\"
    return a - b
# result = add(5, 3)
"""
with open("my_app.py", "w") as f:
    f.write(code_content)
# --- 3. RETRIEVAL: Load and Process the Document ---
print("Loading knowledge base...")
# Load the code file
loader = TextLoader('./my_app.py')
documents = loader.load()
# Split the document into smaller chunks
text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)
docs = text_splitter.split_documents(documents)
```

```
# Create embeddings and store them in a FAISS vector database
embeddings = OpenAIEmbeddings()
db = FAISS.from_documents(docs, embeddings)
print("Knowledge base loaded successfully.")
# --- 4. AUGMENTED GENERATION: Build the RAG Chain ---
# Create a retriever to fetch relevant documents
retriever = db.as_retriever()
# Create the RAG chain that combines the retriever and an LLM
qa_chain = RetrievalQA.from_chain_type(
    llm=OpenAI(),
    chain_type="stuff",
    retriever=retriever,
    return_source_documents=True
)
# --- 5. EXECUTION: Ask a Question ---
question = "How do I add two numbers using a function in this app?"
result = qa_chain({"query": question})
# Print the results
print("\n--- Query ---")
print(question)
print("\n--- Answer ---")
print(result["result"])
```

How the Code Works Step-by-Step

1. **Setup & Knowledge Base:** We start by creating our local knowledge source, the my_app.py file. This file contains the information our agent needs to access.
2. **Load the Document:** The TextLoader reads the contents of my_app.py.
3. **Split into Chunks:** The CharacterTextSplitter breaks the document into smaller pieces. This is crucial because it's more efficient to find specific, relevant chunks than to search the entire file at once.

4. **Create Embeddings & Vector Store:**
    - OpenAIEmbeddings converts each text chunk into a numerical vector (an embedding).
    - FAISS is a local vector database that stores these embeddings for efficient searching. This database now represents our entire knowledge base.
5. **Build the RAG Chain:**
    - The **Retriever** is the component that searches the FAISS database to find the most relevant chunks based on the user's question.
    - RetrievalQA is a pre-built langchain object that links everything together: it takes a question, uses the **retriever** to fetch context, and then sends both to the **LLM** (OpenAI()) to generate an answer.
6. **Ask a Question:** We define our query and pass it to the qa_chain. The chain automatically performs the "retrieve, augment, and generate" steps to get the final answer.